REST ASSURED

Class 4: Authentication, Tokens, Path Parameters & File Upload

Complete beginner-friendly notes with code explanations, real-world scenarios, exercises, and best practices

Based on the uploaded "Rest Assured class 4.txt"

Learning approach

Read the concept first, then type and run the code, inspect the request/response, and finally modify one thing at a time. The local examples use http://localhost:9197 because that base URL appears in the class notes; replace it if your application runs elsewhere.

What you will learn

- Basic, preemptive Basic, Digest, Bearer/JWT, OAuth 2.0 token requests, and OAuth 1.0
- Path parameters and readable method chaining
- Multipart file uploads and DELETE assertions
- End-to-end admin login → category → product → image → cleanup workflow
- Debugging, security practices, and hands-on exercises with solutions

How to use these notes

These notes follow the same topics as the class material. Each topic starts with a plain-English explanation, then code, line-by-line meaning, a realistic scenario, common mistakes, and a practice task. A few source examples are corrected where the original syntax would be fragile or where protocol conventions differ.

Contents

- 1. API authentication: the big picture
- 2. Project setup and imports
- 3. Basic authentication
- 4. Digest authentication
- 5. Bearer token, access token, and JWT
- 6. Getting and refreshing OAuth 2.0 tokens
- 7. OAuth 1.0 and when you may encounter it
- 8. Path parameters
- 9. Method chaining: POST and GET
- 10. Multipart file upload
- 11. DELETE requests and assertions
- 12. End-to-end category and product workflow
- 13. Best practices and common mistakes
- 14. Troubleshooting real failures
- 15. Hands-on exercises
- 16. Solutions and review questions
- 17. Quick revision sheet and references

Important environment note

The example endpoints use a local application at localhost:9197. The workbook cannot confirm that this application is running or that its response codes and payloads match the examples. Run the corresponding backend and adapt endpoint paths, payload fields, expected status codes, and multipart field names to your API contract.

1. API authentication: the big picture

1.1 What does authentication mean?

Authentication answers: “Who are you?” An API may require a username/password or a token before it allows a request. Authorization answers: “What are you allowed to do?” For example, a signed-in user may read products but only an admin may create a category or delete a product.

In an API automation test, you commonly authenticate first, capture the credential issued by the server, and attach it to later requests. The credential may be a Basic header, a Digest challenge response, or a Bearer access token.

1.2 Typical protected API sequence

```
Test client                    API server
   |                               |
   | POST /auth/signin             |
   | username/password or JSON body    |
   |------------------------------->|
   |                               | validate credentials
   | 201/200 + token               |
   |<-------------------------------|
   |                               |
   | GET /products/123             |
   | Authorization: Bearer <token>     |
   |------------------------------->| validate token + permission
   | 200 + product JSON            |
   |<-------------------------------|
```

This sequence is a common pattern, not a universal rule. Some APIs use Basic or Digest auth directly; some use OAuth 2.0 with a separate token endpoint; others use cookies or signed requests.

1.3 Authentication methods at a glance

Method	What is sent	Common use case	Main point
Basic	Base64-encoded username:password in Authorization header	Internal/test endpoints and simple service APIs	Base64 is not encryption; use HTTPS.
Preemptive Basic	Basic credentials sent on the first request	Known Basic-auth endpoint	May avoid a challenge round trip.
Digest	Challenge/response derived from credentials and server nonce	Legacy systems requiring Digest	Server and client must support compatible algorithms.
Bearer token	Authorization: Bearer <token>	Modern protected REST APIs	Protect the token like a password.
OAuth 2.0 token flow	Form-encoded request to authorization server	Service tokens or delegated access	Grant type and client authentication are server-specific.
OAuth 1.0	Signed requests and consumer/access credentials	Legacy third-party integrations	Keep only when the target API requires it.

2. Project setup and imports

2.1 Suggested Maven layout

```
rest-assured-class4/
  pom.xml
  src/
    test/
      java/
        tests/
          AuthenticationTests.java
          ProductApiTests.java
      resources/
        payload/
          AdminLoginPayload.json
          NewCategoryPayload.json
          NewProductPayload.json
          samsung.png
```

Keeping payloads and test code separate is helpful because JSON data can be reviewed or changed without editing Java. Avoid storing real passwords, production tokens, or client secrets in files committed to source control.

2.2 Maven dependencies

Use the versions approved for your project. Keep REST Assured modules on compatible versions, and use the same REST Assured version for core and optional modules. The following is a template; replace version placeholders with the version your project supports.

```
<properties>
    <maven.compiler.release>17</maven.compiler.release>
    <rest-assured.version>6.0.0</rest-assured.version>
    <testng.version>YOUR_APPROVED_TESTNG_VERSION</testng.version>
</properties>

<dependencies>
    <dependency>
        <groupId>io.rest-assured</groupId>
        <artifactId>rest-assured</artifactId>
        <version>${rest-assured.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
```

```
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>${testng.version}</version>
      <scope>test</scope>
    </dependency>
</dependencies>
```

Version reminder

The exact Java, REST Assured, TestNG, and plugin versions depend on your environment. If your organization uses a different Java release, update maven.compiler.release accordingly and use compatible library versions.

2.3 Useful imports

```
import java.io.File;
import java.nio.file.Path;
import java.nio.file.Paths;

import org.testng.Assert;
import org.testng.annotations.Test;

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import io.restassured.specification.RequestSpecification;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;
```

A static import allows you to call given() instead of RestAssured.given(), and equalTo(200) instead of Matchers.equalTo(200). Static imports are optional; use whichever style your team finds clearest.

3. Basic authentication

3.1 Concept

HTTP Basic authentication sends a username/password pair in the Authorization header using Base64 encoding. Base64 is an encoding format, not encryption, so Basic authentication must be used over HTTPS in real environments.

3.2 Challenged Basic authentication

```
given()
    .auth().basic("api_user", "test_password")
.when()
    .get("https://test.example.com/secured/profile")
.then()
    .statusCode(200);
```

Line by line:

- given() starts building the request specification.
- auth().basic(...) configures challenged Basic authentication. REST Assured can wait for the server challenge before resending credentials.
- when().get(...) sends the GET request.
- then().statusCode(200) checks the expected HTTP status. If the actual status differs, the test fails.

3.3 Preemptive Basic authentication

```
given()
    .auth().preemptive().basic(username, password)
.when()
```

```
    .get("https://test.example.com/secured/profile")
.then()
    .statusCode(200);
```

Preemptive Basic sends credentials on the first request. This can avoid an additional challenge round trip. Use it when that behavior matches the application. Use challenged Basic when you specifically need to test the server challenge behavior.

Real-time scenario: internal admin endpoint

An internal inventory service protects a health/configuration endpoint with Basic authentication. Test three cases: valid credentials should return the contract-defined success status; an invalid password should be rejected (commonly 401); and a request with no credentials should be rejected. Do not assume every server uses the same error body.

```
@Test
public void basicAuthRejectsWrongPassword() {
    given()
        .auth().preemptive().basic("api_user", "wrong_password")
    .when()
        .get("https://test.example.com/secured/profile")
    .then()
        .statusCode(401);
}
```

Common mistake

Never print or commit actual usernames/passwords. In automation, load credentials from a secrets manager or environment variables, and use test accounts with the minimum permissions required.

4. Digest authentication

4.1 What is Digest authentication?

Digest authentication is a challenge-response authentication scheme. The server sends a challenge containing values such as a nonce; the client computes a response using the credentials and challenge rather than sending the password in the same way as Basic authentication.

4.2 REST Assured example

```
given()
    .auth().digest(username, password)
.when()
    .get("https://legacy.example.com/secure/report")
.then()
    .statusCode(200);
```

Use Digest only if the target API requires it. Whether a request works depends on the server's supported Digest configuration and algorithms. A failure may be caused by server configuration rather than a Java syntax issue.

Real-time scenario

A company still has an older reporting service protected by Digest. Your regression suite needs to verify that valid users can fetch a report and invalid users are denied. Keep these tests isolated so they do not force newer services to adopt a legacy scheme.

Practice task

Write one positive test expecting the documented success code and one negative test expecting the documented unauthorized response. Check the service contract for whether it responds with 401, 403, or another code in your environment.

5. Bearer token, access token, and JWT

5.1 Definitions

An access token is a credential issued by an authorization/authentication system to access a protected resource. A Bearer token is presented by whoever possesses it. JWT (JSON Web Token) is one possible token format; an access token can also be opaque, so "access token" and "JWT" are not synonyms.

5.2 Preferred REST Assured syntax

```
String accessToken = System.getenv("API_ACCESS_TOKEN");

RestAssured.given()
   .auth().oauth2(accessToken)
.when()
   .get("https://test.example.com/products/28")
.then()
   .statusCode(200)
   .body("id", equalTo(28));
```

REST Assured's oauth2(accessToken) helper adds the token using the usual Bearer header. This makes intent clearer than manually assembling the header.

5.3 Manual Authorization header

```
given()
   .header("Authorization", "Bearer " + accessToken)
.when()
   .get("https://test.example.com/products/28")
.then()
   .statusCode(200);
```

The manual-header version can be useful when an API uses a custom authorization scheme. If the API follows standard Bearer token authentication, prefer auth().oauth2(token).

Real-time test matrix

Test case	Request setup	Expected outcome (confirm contract)
Valid token	Use a current token with required scope	200 for a GET read operation
Missing token	Do not send Authorization header	Usually 401
Expired or malformed token	Send an invalid/expired token	Usually 401
Valid token, insufficient permission	Use a token lacking required scope/role	Often 403
Wrong resource ID	Valid token, nonexistent ID	Often 404

Security warning

Do not use log().all() on requests that contain Authorization headers. Logs may expose access tokens. Tokens should not be printed in console output, test reports, screenshots, or CI artifacts.

6. Getting and refreshing OAuth 2.0 tokens

6.1 Access token flow in plain English

A client sends a request to a token endpoint. The authorization server verifies the client and the requested grant. If approved, it returns fields such as access_token, token_type, expires_in, and sometimes refresh_token. The client uses the access token to call the resource API. If supported and necessary, it may use the refresh token to request a new access token.

```
Test client      Token endpoint      Product API
   | POST /token    |                  |
```

```
   |---------------->|                  |
   | access_token    |                  |
   |<----------------|                  |
   | GET /products/28 (Authorization: Bearer ...)
   |------------------------------------>|
   | 200 + product JSON                  |
   |<------------------------------------|
```

6.2 Service-to-service example: client credentials grant

This flow is commonly used when one service or automated test client accesses another service as itself, rather than signing in as a human user. The actual grant type, token URL, scopes, and client authentication method must match the authorization server contract.

```
String tokenUrl = System.getenv("TOKEN_URL");
String clientId = System.getenv("API_CLIENT_ID");
String clientSecret = System.getenv("API_CLIENT_SECRET");

Response tokenResponse =
   given()
      .auth().preemptive().basic(clientId, clientSecret)
      .contentType(ContentType.URLENC)
      .formParam("grant_type", "client_credentials")
      .formParam("scope", "catalog.read catalog.write")
   .when()
      .post(tokenUrl);

tokenResponse.then().statusCode(200);
String accessToken = tokenResponse.jsonPath().getString("access_token");
Assert.assertNotNull(accessToken, "Token response must include access_token");
Assert.assertFalse(accessToken.trim().isEmpty(), "access_token must not be blank");
```

Why each piece exists:

- TOKEN_URL points to the authorization server, not the resource URL. Keep environment-specific values outside source code.
- ContentType.URLENC communicates that the request body is URL-encoded form data, which is the common OAuth token endpoint format.
- formParam adds form fields to the POST body; it is not the same as queryParam, which adds data to the URL.
- grant_type=client_credentials identifies the grant. The server may require client authentication with HTTP Basic as shown or another method.
- scope asks for the permissions needed by the test. Ask only for what the test requires.
- The test verifies success before extracting access_token, then checks that the token exists and is not blank.

Adapt to your server

Some providers require client_id and client_secret as form fields; others require HTTP Basic or a different client-authentication method. Follow that provider's contract. Do not send credentials in multiple places unless explicitly required.

6.3 Refresh token request

```
Response refreshResponse =
   given()
      .auth().preemptive().basic(clientId, clientSecret)
      .contentType(ContentType.URLENC)
      .formParam("grant_type", "refresh_token")
      .formParam("refresh_token", refreshToken)
   .when()
      .post(tokenUrl);

refreshResponse.then().statusCode(200);
String newAccessToken = refreshResponse.jsonPath().getString("access_token");
Assert.assertNotNull(newAccessToken, "Refresh response must include access_token");
```

Important: the standard OAuth 2.0 refresh grant uses grant_type=refresh_token and the field refresh_token. The camel-case text “refreshToken” in the class notes is not the standard spelling. Some proprietary APIs may define their own contract, so the target server documentation remains authoritative.

6.4 Legacy password grant: recognise it, do not use it for new designs

Your source notes show a username/password grant example. Historically, OAuth 2.0 called this the Resource Owner Password Credentials grant and used grant_type=password, username, password, and optionally scope. Modern OAuth security best practice says this grant must not be used for new designs because it exposes the user’s password to the client. Use a supported modern flow instead—for example Authorization Code with PKCE for user sign-in, or Client Credentials for service-to-service access.

```
// Legacy compatibility example ONLY when the existing test environment
// explicitly requires this flow. Do not introduce it into a new system.
Response tokenResponse =
    given()
        .contentType(ContentType.URLENC)
        .formParam("grant_type", "password")
        .formParam("username", System.getenv("API_USERNAME"))
        .formParam("password", System.getenv("API_PASSWORD"))
        .formParam("scope", "read write")
    .when()
        .post(tokenUrl);

tokenResponse.then().statusCode(200);
String accessToken = tokenResponse.jsonPath().getString("access_token");
```

The original note’s value “credentials/password” is not the standard OAuth grant_type value. If testing a legacy endpoint, use the exact request contract; do not assume every authorization server supports the password grant.

6.5 Important token tests

- Token endpoint success returns a non-empty access_token and a valid token_type.
- Invalid client credentials are rejected.
- Invalid username/password (only for a legacy system that supports that flow) are rejected.
- Invalid scope is denied or reduced according to the API contract.
- Refresh token is rejected when expired, revoked, or invalid.
- Protected endpoint rejects missing/expired tokens and does not leak internal stack traces.

7. OAuth 1.0 and when you may encounter it

OAuth 1.0 uses signed requests and a different protocol model from OAuth 2.0 Bearer tokens. Your class note shows the legacy REST Assured call:

```
given()
    .auth().oauth(consumerKey, consumerSecret, accessToken, tokenSecret)
.when()
    .get("https://legacy.example.com/records")
.then()
    .statusCode(200);
```

Use OAuth 1.0 only when the target system requires it. Do not confuse consumerSecret/tokenSecret in OAuth 1.0 with client_secret/refresh_token in OAuth 2.0. Confirm any extra dependencies required by the REST Assured version in your project.

8. Path parameters

8.1 What is a path parameter?

A path parameter is a variable part of the URL path identifying a specific resource. For example, /category/15 has category ID 15. In REST Assured, define a named placeholder and pass its value with pathParam().

```
int categoryId = 15;

given()
  .pathParam("categoryId", categoryId)
.when()
  .get("http://localhost:9197/category/{categoryId}")
.then()
  .statusCode(200)
  .body("id", equalTo(categoryId));
```

8.2 Path parameter vs query parameter

Type	Example URL	REST Assured method	Usually answers
Path parameter	/products/28	pathParam("productId", 28)	Which exact product?
Query parameter	/products?category=phones&page=2	queryParam("category", "phones")	Filter, search, sort, or paginate which results?
Header	Authorization: Bearer ...	header(...) or auth().oauth2(...)	What metadata or credential accompanies the request?
Body	JSON object in POST/PATCH	body(...)	What resource data should be created or changed?

Small but important source correction

The source shows pathParam("categoryId ", value) with a trailing space in the parameter name. Use exactly "categoryId" with no extra space, and keep the URL placeholder {categoryId} consistent.

8.3 Real-time scenario

After creating a category, the API returns id=42. Save that returned ID and use it when fetching the category. Do not hardcode 42 because another test run may create a different ID.

```
Response createResponse = given()
  .auth().oauth2(accessToken)
  .contentType(ContentType.JSON)
  .body(new File("src/test/resources/payload/NewCategoryPayload.json"))
.when()
  .post("http://localhost:9197/category");

createResponse.then().statusCode(201);
int categoryId = createResponse.jsonPath().getInt("id");

given()
  .auth().oauth2(accessToken)
  .pathParam("categoryId", categoryId)
.when()
  .get("http://localhost:9197/category/{categoryId}")
.then()
  .statusCode(200)
  .body("id", equalTo(categoryId));
```

9. Method chaining: POST and GET

9.1 What is method chaining?

Method chaining links request-building and assertion methods into one readable flow. In REST Assured, a common structure is given() → when() → then().

Part	Purpose	Examples
given()	Set up request details	auth, headers, params, content type, body
when()	Execute the HTTP action	get, post, put, patch, delete
then()	Validate the response	statusCode, header, time, body, log

9.2 POST a product

```
File productPayload = new File(
   "src/test/resources/payload/NewProductPayload.json");

Response response =
   given()
      .auth().oauth2(accessToken)
      .contentType(ContentType.JSON)
      .body(productPayload)
   .when()
      .post("http://localhost:9197/products");

response.then()
   .statusCode(201)
   .body("id", greaterThan(0));

int productId = response.jsonPath().getInt("id");
```

Explanation: authentication attaches the token; contentType(JSON) declares the outgoing body format; body(productPayload) reads the JSON file; post(...) submits the request; then() validates the response; JsonPath extracts the generated product ID for later calls.

9.3 GET a product

```
given()
   .auth().oauth2(accessToken)
   .pathParam("productId", productId)
.when()
   .get("http://localhost:9197/products/{productId}")
.then()
   .statusCode(200)
   .body("id", equalTo(productId))
   .body("title", not(isEmptyOrNullString()));
```

A GET test should validate more than “no exception occurred.” Check the status, the resource ID, and one or more business-critical fields. Only assert fields that are guaranteed by the API contract.

9.4 Logging while chaining

```
given()
   .log().ifValidationFails()
   .auth().oauth2(accessToken)
   .pathParam("productId", productId)
.when()
   .get("http://localhost:9197/products/{productId}")
.then()
   .log().ifValidationFails()
   .statusCode(200);
```

Do not blindly log all data

The class code uses log().all(). This can expose credentials, authorization headers, request payloads with personal data, or access tokens. In normal automation, log only what is needed for diagnostics; mask sensitive headers and avoid logging passwords/tokens altogether.

10. Multipart file upload

10.1 What is multipart/form-data?

Multipart form data lets a request send a file and other form fields in separate parts of one HTTP request. The server typically expects a part name matching its endpoint contract—for example, imageFile. REST Assured supports multiPart() for files, byte arrays, streams, and text.

10.2 Upload a product image

```
Path imagePath = Paths.get(
    System.getProperty("user.dir"),
    "src", "test", "resources", "payload", "samsung.png");
File imageFile = imagePath.toFile();
Assert.assertTrue(imageFile.isFile(), "Image test file was not found: " + imagePath);

Response uploadResponse =
    given()
        .auth().oauth2(accessToken)
        .pathParam("productId", productId)
        .multiPart("imageFile", imageFile, "image/png")
    .when()
        .post("http://localhost:9197/products/{productId}/image");

uploadResponse.then()
    .statusCode(200); // Change this to the documented success code for your API.
```

The file is located using a project-relative path built from the repository root. isFile() checks that the test will not silently send a missing file. multiPart("imageFile", file, "image/png") names the field and declares its media type. The server may return 200, 201, or another documented success code.

Multipart boundary

Usually let REST Assured set the multipart content type and boundary. Do not manually set Content-Type: multipart/form-data unless you know how the boundary is being generated; a missing or incorrect boundary can make the request invalid.

10.3 Real-time upload test cases

Scenario	What to verify
Valid PNG/JPEG	Success status; response identifies the correct product and uploaded image.
Missing local file	Fail the test setup clearly before making the HTTP request.
Unsupported extension/MIME type	Server rejects according to its documented validation response.
File too large	Server returns documented size-limit error.
Invalid product ID	Upload is rejected; no orphaned file/resource should be created.
No/expired token	Upload is unauthorized; resource is not modified.

10.4 Best practices for file tests

- Use small, safe fixture files checked into test resources; avoid private or copyrighted customer data.
- Assert the upload response, and when possible perform a follow-up GET to verify image metadata or URL.
- Use unique filenames when uploads can collide.
- Ensure cleanup removes created products and uploaded assets when the API supports deletion.
- Keep MIME type, file size, and accepted extensions aligned with the server contract.

11. DELETE requests and assertions

11.1 Chained DELETE example

```
given()
   .auth().oauth2(accessToken)
   .pathParam("productId", productId)
.when()
   .delete("http://localhost:9197/products/{productId}")
.then()
   .statusCode(204)
   .time(lessThan(5000L));
```

This example assumes the API contract uses 204 No Content for a successful deletion. The 5-second assertion is illustrative; set thresholds using your service-level expectation and test environment. Do not enforce an arbitrary threshold that produces flaky results under CI load.

11.2 Why not assert every header after DELETE?

A 204 response normally has no response body. The server may omit Content-Type because no representation was returned. Therefore, asserting Content-Type contains text/plain is not generally valid unless your API contract explicitly requires that header. Also, exact status-line wording can vary by HTTP version/server; statusCode(204) is usually the more stable assertion.

11.3 Verify deletion with a follow-up request

```
given()
   .auth().oauth2(accessToken)
   .pathParam("productId", productId)
.when()
   .get("http://localhost:9197/products/{productId}")
.then()
   .statusCode(404); // if the API contract specifies 404 after deletion
```

A follow-up GET can demonstrate that the resource is no longer available. Some APIs return 410 Gone or use soft-delete semantics, so confirm the expected behavior first.

Real-time scenario: inventory cleanup

An automated test creates a product, verifies its fields, uploads an image, and then deletes the product. Cleanup matters because test data left behind can pollute later tests and cause duplicate records, quota issues, or misleading reports.

12. End-to-end category and product workflow

The source class demonstrates this chain: log in as admin; capture the token; create a category; capture its ID; fetch the category; create a product; capture its ID; fetch the product; upload an image; delete the product. The following example keeps the dependency explicit in one end-to-end test so a beginner can see where each value comes from.

Adapt this example before running

The login response code and JSON file shapes must match your local API. This example expects login to return a token field and create endpoints to return id fields. It uses 201 for creation, 200 for reads/uploads, and 204 for deletion as examples only. Change each expectation to the contract for your service.

12.1 Example TestNG workflow

```
package madeEasyTutorial;

import java.io.File;
import java.nio.file.Paths;
```

```
import org.testng.Assert;
import org.testng.annotations.Test;

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;

import static org.hamcrest.Matchers.*;

public class AdminProductWorkflowTest {

    private static final String BASE_URL =
        System.getProperty("baseUrl", "http://localhost:9197");

    @Test
    public void adminCanCreateReadUploadAndDeleteProduct() {
        // 1. Login and acquire token
        File loginPayload = Paths.get(
            "src", "test", "resources", "payload", "AdminLoginPayload.json")
            .toFile();
        Assert.assertTrue(loginPayload.isFile(), "Login payload not found");

        Response login = RestAssured.given()
            .contentType(ContentType.JSON)
            .body(loginPayload)
        .when()
            .post(BASE_URL + "/auth/signin");

        login.then().statusCode(201); // Use the documented login status.
        String token = login.jsonPath().getString("token");
        Assert.assertNotNull(token, "Login response must contain token");
        Assert.assertFalse(token.trim().isEmpty(), "Token must not be blank");

        // 2. Create a category
        File categoryPayload = Paths.get(
            "src", "test", "resources", "payload", "NewCategoryPayload.json")
            .toFile();
        Assert.assertTrue(categoryPayload.isFile(), "Category payload not found");

        Response createCategory = RestAssured.given()
            .auth().oauth2(token)
            .contentType(ContentType.JSON)
            .body(categoryPayload)
        .when()
            .post(BASE_URL + "/category");

        createCategory.then().statusCode(201);
        int categoryId = createCategory.jsonPath().getInt("id");
        Assert.assertTrue(categoryId > 0, "Created category must have a positive id");

        // 3. Read the category using the ID returned by creation
        RestAssured.given()
            .auth().oauth2(token)
            .pathParam("categoryId", categoryId)
        .when()
            .get(BASE_URL + "/category/{categoryId}")
        .then()
            .statusCode(200)
            .body("id", equalTo(categoryId));

        // 4. Create a product. Ensure its payload has any category reference
        // required by your API (for example, categoryId).
        File productPayload = Paths.get(
            "src", "test", "resources", "payload", "NewProductPayload.json")
            .toFile();
        Assert.assertTrue(productPayload.isFile(), "Product payload not found");

        Response createProduct = RestAssured.given()
```

```
            .auth().oauth2(token)
            .contentType(ContentType.JSON)
            .body(productPayload)
        .when()
            .post(BASE_URL + "/products");

        createProduct.then().statusCode(201);
        int productId = createProduct.jsonPath().getInt("id");
        Assert.assertTrue(productId > 0, "Created product must have a positive id");

        // 5. Read the newly created product
        RestAssured.given()
            .auth().oauth2(token)
            .pathParam("productId", productId)
        .when()
            .get(BASE_URL + "/products/{productId}")
        .then()
            .statusCode(200)
            .body("id", equalTo(productId));

        // 6. Upload a product image
        File image = Paths.get(
            "src", "test", "resources", "payload", "samsung.png").toFile();
        Assert.assertTrue(image.isFile(), "Image test file not found: " + image);

        RestAssured.given()
            .auth().oauth2(token)
            .pathParam("productId", productId)
            .multiPart("imageFile", image, "image/png")
        .when()
            .post(BASE_URL + "/products/{productId}/image")
        .then()
            .statusCode(200); // Adapt to documented API response.

        // 7. Delete the product
        RestAssured.given()
            .auth().oauth2(token)
            .pathParam("productId", productId)
        .when()
            .delete(BASE_URL + "/products/{productId}")
        .then()
            .statusCode(204);

        // Optional: delete the category too if your API allows and the
        // product/category relationship no longer prevents deletion.
    }
}
```

12.2 Walkthrough of the workflow

1. Login returns a token. The test checks the login status before reading token; otherwise a failed login could cause confusing downstream failures.
2. Create category returns a server-generated ID. The next request uses that ID instead of hardcoding a value.
3. Create product should use a payload consistent with the category relationship expected by your API. The sample payload in your project may need to include categoryId or a nested category object.
4. Upload uses multipart/form-data, not a normal JSON request body. The field name imageFile must match the server implementation.
5. Delete asserts the documented status. A follow-up GET can verify deletion when the API contract says the deleted item is no longer found.

12.3 Important limitation in the original class design

The source stores token, categoryId, and productId in instance fields and relies on @Test(priority=...). Priorities can control execution order, but they do not make tests independent, and parallel execution or a failure earlier in the chain can invalidate

later tests. Use explicit TestNG dependsOnMethods where appropriate, a setup method, or a single end-to-end test for a genuinely dependent workflow. Keep independent read/negative tests separate.

13. Best practices and common mistakes

13.1 Authentication and secrets

- Use HTTPS outside local development. Basic credentials and Bearer tokens are sensitive secrets.
- Load secrets from environment variables or a secrets manager, not literals in Java source or committed JSON files.
- Never log full authorization headers or token responses. Avoid log().all() around authenticated calls.
- Use a least-privilege test account. An image upload test should not need unrestricted production administration.
- Use supported OAuth flows. Do not introduce the Resource Owner Password Credentials grant into a new design.
- Never reuse personal/production passwords in test automation.

13.2 Robust test design

- Assert status code before extracting IDs or tokens.
- Validate important response fields, not only status codes.
- Use pathParam for resource identifiers in URL paths; use queryParam for filters and pagination.
- Build file paths with Paths.get(...) and verify the file exists before sending.
- Use response values (created ID, token) instead of hardcoded generated data.
- Keep tests repeatable: prepare known inputs and clean up resources that the test creates.
- Use explicit dependency or orchestration for tests that require previous data; do not rely on execution order accidentally.
- Use reasonable response-time expectations based on service goals, not a value guessed from one local run.
- Assert headers only when the API contract guarantees them. Do not assume DELETE 204 responses contain Content-Type.
- Use configuration for base URLs and never hardcode a production endpoint in a local learning test.

13.3 Logging and diagnostics

Approach	When it fits	Caution
No request/response log	Normal successful runs	Keep concise test result output.
Log on validation failure	Most automated regression runs	Ensure secrets are masked.
Full log	Temporary local debugging only	Never include passwords/tokens or private data in shared logs.
Explicit diagnostic fields	CI pipelines and reports	Print request ID, status code, duration, and a safe error snippet.

13.4 Recommended assertion pattern

```
Response response = given()
    .auth().oauth2(accessToken)
    .pathParam("productId", productId)
.when()
    .get(BASE_URL + "/products/{productId}");

response.then()
    .statusCode(200)
    .contentType(ContentType.JSON)
    .time(lessThan(5000L))
    .body("id", equalTo(productId))
    .body("title", not(isEmptyOrNullString()));
```

Use only the content-type and time assertions that your service contract and performance target justify. The title matcher expects a string-like, non-empty value; if your API permits an empty title, that assertion would be incorrect.

14. Troubleshooting real failures

Symptom	Possible reason	What to check
401 Unauthorized	Wrong credentials/token, token expired, token not attached	Inspect safe auth configuration; verify scheme is Bearer; verify token endpoint result without printing the token.
403 Forbidden	Authenticated but insufficient role/scope	Check scopes/roles and whether endpoint is admin-only.
404 Not Found	Wrong base URL/path, missing resource, ID not created	Check endpoint path, response ID, and environment.
400 Bad Request	Wrong payload fields, content type, missing form fields	Compare body with the API contract; ensure JSON vs URLENC is correct.
415 Unsupported Media Type	Body format or Content-Type is wrong	JSON endpoint: application/json; OAuth form endpoint: application/x-www-form-urlencoded; file upload: multipart with boundary.
File not found	Path assembled from incorrect working directory	Use Paths.get(user.dir, ...); assert file.isFile() before request.
Multipart upload rejected	Wrong part name, unsupported MIME type, bad boundary	Check part name, MIME type, size limit, and let REST Assured create boundary.
Null token / later 401s	Login request failed before token extraction	Assert login response status and token field before calling protected endpoints.
Flaky response-time check	Threshold too strict or environment load varies	Use service-level performance goals; use dedicated performance tests for latency distribution.
TestNG product test fails alone	Test relies on a token/productId created by another test	Make setup explicit or create data in the same test; remove hidden order dependencies.

Debugging sequence

6. Confirm the correct environment is running and the base URL is reachable.
7. Check the HTTP method, path, path parameter value, and request content type.
8. Check whether authentication is required and whether the token has the right format/scope.
9. Compare the response status and safe error message with the API contract.
10. Check generated IDs, fixture paths, and whether prior test steps completed.
11. Re-run the smallest failing test in isolation; then restore it to the whole suite.

15. Hands-on exercises

Try each task first without looking at the solution section. Use a local/test system that you are authorized to access. Do not send practice credentials to a real production system.

Exercise 1 — Basic authentication

Write a GET test that calls a protected endpoint using challenged Basic authentication and checks for status 200. Then write a negative test using an incorrect password and the expected unauthorized status.

Exercise 2 — Preemptive Basic

Rewrite Exercise 1 using preemptive Basic. Explain one reason to choose this mode and one reason you may deliberately test challenged Basic instead.

Exercise 3 — Digest

Write a Digest-authenticated GET request to a fictional legacy report endpoint. Add the success assertion and note what server details you would check if it failed.

Exercise 4 — Bearer token

Read the access token from an environment variable, call GET /products/28 with oauth2(token), and verify the status and response id.

Exercise 5 — Token endpoint

Build a client-credentials token request using a URL-encoded body. Assert token endpoint success and verify access_token exists and is not blank. Do not log the response token.

Exercise 6 — Refresh token

Modify the token request to use grant_type=refresh_token and refresh_token=<saved refresh token>. Verify the new access token. Explain why refresh tokens must be protected carefully.

Exercise 7 — Path parameter

Fetch a category using pathParam("categoryId", categoryId). Add an assertion that the response id equals the exact ID passed in the URL.

Exercise 8 — Product POST method chaining

Read NewProductPayload.json, create a product with a Bearer token, assert the documented creation status, and extract a positive generated id.

Exercise 9 — Multipart upload

Upload samsung.png as the multipart part named imageFile. Validate the local fixture exists first and assert the server-documented success response. Try a file with the wrong type in a separate negative test.

Exercise 10 — DELETE

Delete a product using a path parameter. Assert 204 only if the API contract uses No Content. Explain why checking Content-Type may be inappropriate for a 204 response.

Exercise 11 — Fix the class sample

Identify at least four reliability risks in a suite that calls login, create category, GET category, create product, upload image, and delete product using @Test(priority=1..8), instance fields, and log().all(). Propose fixes.

Exercise 12 — End-to-end workflow

Implement login → create category → GET category → create product → GET product → upload image → DELETE product. Use server-generated IDs, meaningful assertions, protected secrets, and cleanup where possible.

16. Solutions and review questions

Solution 1 — Basic auth

```
given()
   .auth().basic("api_user", System.getenv("API_PASSWORD"))
.when()
   .get("https://test.example.com/secured/profile")
.then()
   .statusCode(200);
```

For the negative version, supply a deliberately invalid test password and assert the API's documented rejection status, commonly 401. Never store actual passwords in the test source.

Solution 2 — Preemptive Basic

```
given()
   .auth().preemptive().basic("api_user", System.getenv("API_PASSWORD"))
.when()
   .get("https://test.example.com/secured/profile")
.then()
   .statusCode(200);
```

Preemptive Basic sends the credentials immediately, which can avoid a challenge round trip. Challenged Basic is useful when verifying that the server issues and handles its authentication challenge.

Solution 4 — Bearer token

```
String token = System.getenv("API_ACCESS_TOKEN");
Assert.assertNotNull(token, "Set API_ACCESS_TOKEN before running");
Assert.assertFalse(token.trim().isEmpty(), "Token must not be blank");

given()
   .auth().oauth2(token)
.when()
   .get("https://test.example.com/products/28")
.then()
   .statusCode(200)
   .body("id", equalTo(28));
```

Solution 5 — Token endpoint

```
Response response = given()
   .auth().preemptive().basic(
      System.getenv("API_CLIENT_ID"),
      System.getenv("API_CLIENT_SECRET"))
   .contentType(ContentType.URLENC)
   .formParam("grant_type", "client_credentials")
   .formParam("scope", "catalog.read")
.when()
   .post(System.getenv("TOKEN_URL"));

response.then().statusCode(200);
String token = response.jsonPath().getString("access_token");
Assert.assertNotNull(token);
Assert.assertFalse(token.trim().isEmpty());
```

Solution 6 — Refresh token

```
Response response = given()
   .auth().preemptive().basic(
      System.getenv("API_CLIENT_ID"),
      System.getenv("API_CLIENT_SECRET"))
   .contentType(ContentType.URLENC)
   .formParam("grant_type", "refresh_token")
   .formParam("refresh_token", refreshToken)
.when()
   .post(System.getenv("TOKEN_URL"));

response.then().statusCode(200);
String renewedToken = response.jsonPath().getString("access_token");
Assert.assertNotNull(renewedToken);
```

A refresh token is typically longer-lived than an access token and can be used to obtain additional access tokens; if stolen, it may extend an attacker's ability to access the API. Protect it like a secret and follow the authorization server's rotation/revocation rules.

Solution 7 — Path parameter

```
given()
   .auth().oauth2(accessToken)
   .pathParam("categoryId", categoryId)
.when()
   .get(BASE_URL + "/category/{categoryId}")
.then()
   .statusCode(200)
   .body("id", equalTo(categoryId));
```

Solution 8 — Create product

```
Response createResponse = given()
   .auth().oauth2(accessToken)
   .contentType(ContentType.JSON)
   .body(new File("src/test/resources/payload/NewProductPayload.json"))
.when()
   .post(BASE_URL + "/products");

createResponse.then().statusCode(201);
int productId = createResponse.jsonPath().getInt("id");
Assert.assertTrue(productId > 0);
```

Solution 9 — Multipart file upload

```
File image = new File("src/test/resources/payload/samsung.png");
Assert.assertTrue(image.isFile(), "Image file must exist");

given()
   .auth().oauth2(accessToken)
   .pathParam("productId", productId)
   .multiPart("imageFile", image, "image/png")
.when()
   .post(BASE_URL + "/products/{productId}/image")
.then()
   .statusCode(200); // Replace with the actual documented success code.
```

Solution 10 — DELETE

```
given()
   .auth().oauth2(accessToken)
   .pathParam("productId", productId)
.when()
   .delete(BASE_URL + "/products/{productId}")
.then()
   .statusCode(204);
```

A 204 response says the operation succeeded and no response representation is provided. Assert Content-Type only if the API explicitly specifies one for this response.

Solution 11 — Risks and improvements

- Risk: tests depend on priorities and shared fields. Improvement: use explicit dependencies for linked tests or keep one true end-to-end workflow together.
- Risk: login token is extracted without checking status/token presence. Improvement: assert login status and nonblank token first.
- Risk: log().all() may expose credentials or tokens. Improvement: safe, failure-only logging with secret masking.
- Risk: product/category IDs are mutable shared state. Improvement: derive IDs from responses and keep lifecycle scope explicit.

- Risk: hardcoded local URL. Improvement: configurable base URL per environment.
- Risk: files may not exist on CI agents. Improvement: project-relative paths, isFile() checks, and committed safe fixtures.
- Risk: headers/status line assertions may be brittle. Improvement: assert API-contract fields and status codes; avoid undocumented headers.
- Risk: created test data may remain after a failed test. Improvement: use cleanup and avoid destructive tests against production.

Review questions

12. What is the difference between Basic and Bearer authentication?
13. What makes preemptive Basic different from challenged Basic?
14. Why should an access token not be included in logs?
15. What is the difference between an access token and a JWT?
16. What values should the standard OAuth 2.0 refresh request use for grant_type and refresh_token?
17. When do you use pathParam versus queryParam?
18. Why is application/x-www-form-urlencoded used by many token endpoints?
19. Why should REST Assured normally create the multipart boundary?
20. Why is statusCode(204) often better than asserting a status-line string?
21. Why are @Test(priority=…) and shared instance fields a risk for independent automation tests?

17. Quick revision sheet

Need	Code pattern
Basic auth	auth().basic(username, password)
Preemptive Basic	auth().preemptive().basic(username, password)
Digest	auth().digest(username, password)
Bearer token	auth().oauth2(accessToken)
OAuth token request	contentType(ContentType.URLENC).formParam("grant_type", "client_credentials")
Refresh token	formParam("grant_type", "refresh_token").formParam("refresh_token", refreshToken)
Path variable	pathParam("productId", productId) + "/products/{productId}"
JSON body	contentType(ContentType.JSON).body(payloadFile)
Multipart file	multiPart("imageFile", imageFile, "image/png")
Assert response	then().statusCode(200).body("id", equalTo(id))
Delete with no body	then().statusCode(204)
Safe diagnostics	failure-only logs; never dump credentials or tokens

Before you run your tests

- Confirm that your local backend is running at the configured base URL.
- Confirm login, create, read, upload, and delete status codes in API documentation.
- Check that JSON fixture files contain valid schema and any required category reference.
- Set secrets as environment variables; do not paste real values into the code.
- Run the smallest authentication test, then the create/read/upload/delete workflow.
- Inspect failures and clean up data before rerunning.

References and further reading

The main subject order and local endpoint examples in this workbook come from the uploaded “Rest Assured class 4.txt”. The notes below add official references for REST Assured authentication/multipart syntax and OAuth token conventions.

- **REST Assured Usage Guide:** https://github.com/rest-assured/rest-assured/wiki/Usage
  Official usage documentation for authentication helpers, OAuth 2.0, and multipart form data.
- **RFC 6749 — The OAuth 2.0 Authorization Framework:** https://www.rfc-editor.org/rfc/rfc6749.html
  Defines standardized token endpoint parameters, access tokens, refresh tokens, scopes, and grant types.

- **RFC 9700 — Best Current Practice for OAuth 2.0 Security:** https://www.rfc-editor.org/rfc/rfc9700.html
  Current OAuth security recommendations; states that the Resource Owner Password Credentials grant must not be used.

Final takeaway

Authentication is not only syntax. A reliable test verifies that the correct credential is issued, sent only to the intended endpoint, rejected when invalid, and never exposed in logs. Build the test around the API contract and make the resource lifecycle explicit.